Development of a Segmented, Reliable Communication Protocol for Data Transmission over Long-Range (LoRa) Links

Alessandro Monticelli

***Abstract*—This paper presents LAMP (Long-Range Adaptive Multi-Packet Protocol), a lightweight transport architecture that enables the continuous transfer of large data blocks over point-to-point Long Range (LoRa©) networks. To overcome the severe Time-on-Air overhead and duty-cycle exhaustion typical of existing Stop-and-Wait solutions, the proposed architecture segments payloads into up to 256 chunks per transmission window. Each chunk is secured by a compact 9-byte Logical Header incorporating IPv6-ready node addressing and a CRC-16 checksum. The protocol provides four distinct operational modes combining connection state (connectionless vs. stateful) and reliability (best-effort vs. SACK-ARQ feedback). By requiring receiver feedback only at sequence boundaries rather than per-packet, the SACK mechanism significantly reduces acknowledgment overhead without relying on heavy Forward Error Correction. We outline a comprehensive testing methodology to evaluate the protocol across variable distances (1–5000 m) under Line-of-Sight and Non-Line-of-Sight conditions. Initial short-range benchmarking demonstrates that the reliable mode introduces minimal latency and throughput overhead compared to the best-effort baseline, providing a flexible, high-capacity data link for resource-constrained embedded systems.**

***Index Terms*—LoRa, selective acknowledgment, packet fragmentation, transport protocol, Internet of Things.**

I. Introduction

LOW-POWER, low-cost and long-range wireless communication technologies are becoming increasingly important in the context of the Internet of Things (IoT). Applications such as environmental monitoring, precision agriculture or Unmanned Aerial Vehicles (UAVs), require reliable data transmission over extended distances with strict power and hardware constraints [1].

A range of different communication standards, including ZigBee, Sigfox and LoRa© have been adopted to address these needs, and LoRa©, in particular, has emerged as the leading choice for low-power and long-range deployments thanks to its robust modulation technique and open network architecture [2]. However, conventional solutions based on LoRa© (particularly those following the Long Range Wide Area Network (LoRaWAN©) specifications) are optimized for small, non-frequent payloads [3].

This layout limitation poses severe restrictions on modern IoT applications, such as drone telemetry, environmental log collections, or remote imaging, which demand the continuous transfer of large data blocks [4], [5]. Standard LoRaWAN links require substantial transmission overhead, restrictive duty-cycle limitations, and lack optimized mechanisms for handling large, multi-packet payloads in point-to-point configurations.

This paper presents the design and implementation of a comprehensive, lightweight transport protocol for point-to-point LoRa© networks. While early iterations of this work primarily focused on payload segmentation, the introduction of a Selective Acknowledgment (SACK) scheme, a Hardware Abstraction Layer (HAL), and dual reliable/best-effort transmission modes has evolved the architecture into a full-fledged transport solution. It is designed to support the transmission of large data payloads (up to approximately 62.7 KB) over point-to-point topologies, theoretically minimizing the radio duty-cycle footprint compared to traditional Stop-and-Wait approaches while operating entirely independently of higher-layer middleware (e.g., LoRaWAN©).

The key contributions of this work are three-fold:

- **High-Efficiency SACK Protocol**: Unlike existing LP-WAN multimedia transmission protocols in literature that rely on Stop-and-Wait (S&W) ARQ, which requires an acknowledgment for every single packet and introduces severe latency and duty-cycle exhaustion, the proposed protocol implements a Selective Acknowledgment (SACK) scheme. By utilizing a compact, cumulative binary bitmap inside a single feedback packet, the receiver reports the status of up to 255 chunks in a single round-trip, minimizing over-the-air overhead and transceivers' duty-cycle footprint.
- **Lightweight Packet Segmentation and Abstraction**: The protocol introduces a 9-byte logical header (with 8-bit IPv6-ready node addressing) and a custom CRC-16 integrity check on partial payloads, allowing robust reassembly of out-of-order chunks on resource-constrained microcontrollers.
- **Modular C++17 Implementation and TDD Validation**: The core state machine is completely decoupled from the physical radio driver through a Hardware Abstraction Layer (HAL). This design enables both native host-based unit testing (using the Unity framework) and physical deployment on Semtech SX126x transceivers, bridging the gap between simulation and real-world embedded systems.

This approach is generalizable to a wide range of IoT applications requiring extended data capacity over extended distances and high power efficiency, such as drone telemetry, marine biologging, or multiagent coordinated communications.

II. State of The Art

Reliable packet transmission and fragmentation over Low-Power Wide-Area Networks (LPWANs) have been actively studied to support data-intensive IoT applications.

A prominent industrial standard is Static Context Header Compression (SCHC) [5], which introduces a framework for fragmenting IPv6 packets over resource-constrained networks. To guarantee reliability, SCHC ACK frames carry a bitmap representing the status of individual tiles within a window (ACK-on-Error). However, SCHC is designed as a generic adaptation layer for IP-based architectures, requiring pre-shared context rules and gateway-centric networks. Similarly, the LoRa Alliance standardized the Fragmented Data Block Transport specification to support Firmware Update Over The Air (FUOTA) using Forward Error Correction (FEC) codes to reconstruct blocks without feedback. However, FEC-based recovery introduces a constant over-the-air encoding overhead and high computational requirements for low-cost embedded systems.

For custom peer-to-peer LoRa networks, early efforts focused on overcoming the severe airtime latencies of traditional Stop-and-Wait (S&W) ARQ schemes [4], where the transmitter blocks and waits for a dedicated acknowledgment after every single chunk. A pioneering contribution in this domain was introduced by Chen et al. [6] with the Multi-Packet LoRa (MPLR) protocol. MPLR introduced the core concept of replacing Stop-and-Wait ARQ with a windowed batch transmission scheme acknowledged by a bit-vector acknowledgment packet (`BVACK`). Subsequent specialized studies explored multi-hop relay networks for visual IoT coverage [7], comprehensive mesh survey frameworks [8], and Time Division Multiplexing (TDM) scheduled channels [9].

While `LAMP` draws foundational inspiration from the bit-vector SACK mechanism introduced in MPLR [6], our architecture introduces major engineering and protocol-level advancements that generalize and optimize the concept for modern embedded IoT applications:

- **Header Overhead Optimization**: MPLR relies on a 16-byte header per fragment (carrying 4-byte Destination EUI, 4-byte Source EUI, Service Number, Sequence Number, Flags, Payload Size, Batch Size, and Checksum). In contrast, `LAMP` reduces header overhead to a 9-byte Logical Header (1-byte `srcAddr`, 1-byte `dstAddr`, 2-byte `messageId`, 1-byte `totalChunks`, 1-byte `chunkIndex`, 1-byte `payloadSize`, 1-byte `flags`, and 1-byte `protocolVersion`). This saves 7 bytes of over-the-air overhead per chunk.
- **Target Scope**: MPLR was engineered specifically as an application-level image transmission protocol for agricultural uplink nodes to a central gateway. `LAMP` is designed as a domain-agnostic, general-purpose transport layer capable of segmenting any arbitrary data payload up to 62.7 KB (ranging from high-frequency sensor telemetry arrays and robotic SLAM keyframes to firmware binaries and multiagent state matrices).
- **Flexible Operational Modes**: MPLR operates under a single stateful batch delivery model. In contrast, `LAMP` provides four operational modes spanning *Unreliable Connectionless* (best-effort datagrams), *Unreliable Connected* (session-tracked streaming), *Reliable Connectionless* (stateless SACK-ARQ), and *Reliable Connected* (stateful negotiated streams).
- **Dynamic Parameter Negotiation**: MPLR utilizes a fixed batch size and requires a 4-way handshake tear-down (`FIN/ACK`). `LAMP` implements a lightweight 3-Way Handshake (3WHS) embedded with dynamic parameter negotiation (`SynMetadata` and `SynAckMetadata` negotiating chunk size, sliding window, and inactivity timeout) and replaces explicit tear-down overhead with automatic idle session pruning.

To clearly position the proposed protocol within the current technological landscape, Table I provides an architectural comparison with other prominent fragmentation and transport solutions over LoRa, including MPLR.

III. Proposed System Architecture

The proposed protocol, `LAMP` (**L**ong-range **A**daptive **M**ulti-packet **P**rotocol), operates as a custom transport-layer architecture running directly over LoRa© physical layers. It aims to segment arbitrarily large payloads at the transmitter and reliably reconstruct them at the receiver, while keeping Logical Header and over-the-air (OTA) overhead minimal.

A. Protocol Evolution: From v1 Baseline to v2 Stateful Architecture

The development of `LAMP` followed an iterative engineering trajectory, progressing from an initial stateless baseline (v1) to a full-fledged, stateful, and addressed transport framework (v2).

1) Baseline Architecture (v1: Stateless 7-Byte Header): The initial protocol version (v1) was engineered for maximum header compression during periodic sensor telemetry transfers over point-to-point links. It utilized a compact 7-byte Logical Header containing only sequence tracking and payload bounds (`messageId` [2 B], `totalChunks` [1 B], `chunkIndex` [1 B], `payloadSize` [1 B], `flags` [1 B], and `protocolVersion` [1 B]).

While v1 achieved high spectral efficiency for isolated datagram bursts, real-world deployment on embedded hardware exposed three critical architectural vulnerabilities:

- **Unchecked Buffer Pre-allocation**: Receivers were forced to allocate reassembly memory speculatively upon receiving the first fragment of an unannounced stream. When receiving large payloads (e.g., image tiles or firmware blocks), memory-constrained microcontrollers frequently suffered heap exhaustion or buffer overflows.
- **Session Collision and Hijacking**: Lacking explicit source and destination identifiers, multiple unaddressed transmitters sharing the same RF channel risked interleaving fragments with identical sequence numbers, leading to corrupted reassembly buffers.

TABLE I
ARCHITECTURAL COMPARISON OF LORA TRANSPORT SOLUTIONS

Feature	Stop-and-Wait	SCHC [5]	LoRaWAN Frag.	MPLR [6]	Proposed (LAMP)
ACK / Retransmission	Per-Packet ACK	ACK-on-Error Bitmap	None (FEC Code)	Bit-Vector SACK	Bit-Vector SACK
Header Size	2–4 Bytes	1–2 Bytes	2 Bytes	16 Bytes	9 Bytes
Max Frame Payload	240+ Bytes	Variable	Variable	239 Bytes	244 Bytes
Operational Modes	1 (Reliable S&W)	2 (ACK / No-ACK)	1 (Unreliable FEC)	1 (Reliable Batch)	4 Operational Modes
Handshake / Setup	None	Static Rules	Session Setup	3WHS + 4WHS Teardown	3WHS Dynamic Negotiation
Target Scope	Simple P2P	IP Star Network	Gateway FUOTA	Image Upload	General-Purpose P2P / Multi-Node

- **Static Transport Parameters**: Fixed chunk sizes and static timeout thresholds prevented dynamic adaptation to link quality degradation or variable receiver memory limits.

2) Complete Architecture (v2: Stateful 9-Byte Header with 3WHS): To resolve the limitations of v1, LAMP Version 2 introduces a complete, production-grade transport architecture. Version 2 expands the Logical Header to 9 bytes by embedding explicit 8-bit Source (srcAddr) and Destination (dstAddr) Node Identifiers, while incorporating an integrated 3-Way Handshake (3WHS) mechanism.

Although adding 2 bytes to the header increases over-the-air frame length by less than 0.8% on a 244-byte payload, this trade-off delivers essential architectural guarantees:

1) **Guaranteed Memory Safety**: The 3WHS exchange allows nodes to negotiate reassembly buffer limits (SynMetadata) prior to payload transmission, guaranteeing zero buffer overflows.
2) **Session Isolation**: Explicit node addressing completely prevents promiscuous packet hijacking and session corruption in multi-node wireless topologies.
3) **IPv6 Readiness**: The 8-bit address space maps natively to local IPv6 link-local subnet suffixes without the overhead of full 128-bit IP headers over the air.

B. Operational Modes

To accommodate diverse IoT workload requirements—ranging from ultra-fast periodic telemetry streams to critical high-capacity firmware or imaging transfers—LAMP defines four distinct operational modes across two primary dimensions: *Reliability* (Unreliable Best-Effort vs. SACK-Reliable ARQ) and *Connection State* (Connectionless Stateless vs. Connection-Oriented Stateful).

C. OTA Packet Structure and Header Optimization

Each individual packet transmitted over the air is structured sequentially as a fixed 9-byte Logical Header, a variable-length Application Payload (up to 244 bytes), and a 2-byte CRC footer. Unused padding bytes at the end of partial payloads are omitted from over-the-air transmission to minimize airtime and regulatory duty-cycle footprint.

1) Implicit Boundary Detection and Sequence Indexing: To maintain a compact 9-byte header while supporting robust out-of-order packet reassembly, LAMP avoids dedicated boundary control bits. Instead, Start-of-Message (SOM) and End-of-Message (EOM) boundaries are implicitly derived at the receiver directly from the 0-based fragment index (chunkIndex) and the total fragment count (totalChunks):

$$\text{SOM} \iff \texttt{chunkIndex} == 0 \tag{1}$$

$$\text{EOM} \iff \texttt{chunkIndex} == \texttt{totalChunks} - 1 \tag{2}$$

Explicitly carrying totalChunks in every fragment header allows the receiver to instantly determine the exact message bounds and pre-allocate the required reassembly buffer and SACK bitmap upon receiving *any* fragment, even if fragments arrive out of order or if the initial chunk is lost.

2) Compact Node Addressing: To support multi-node communication without incurring the prohibitive airtime overhead of full 128-bit IPv6 headers on LoRa PHY payloads, LAMP introduces a compact 8-bit Node Identifier space:

- **0x00 (ADDRESS_UNASSIGNED)**: Reserved for legacy or unaddressed P2P datagrams (promiscuous mode).
- **0x01–0xFE**: 254 unique individual node addresses (Unicast).
- **0xFF (ADDRESS_BROADCAST)**: Broadcast address processed by all active receiving nodes on the RF channel.

This 8-bit Node ID design is IPv6-ready: at an edge gateway or application layer, a local 8-bit Node ID (e.g., 0x42) maps directly to an IPv6 link-local subnet suffix (e.g., fe80::42), enabling seamless IP-layer integration without transmitting heavy IPv6 headers over the constrained radio link.

The exact byte alignment and offsets of the 9-byte header packet structure are presented in Table III and Figure 1.

The fields in the Logical Header are defined as follows:

- **srcAddr (1 byte)**: Source node identifier (0x01–0xFE, or 0x00 for unassigned).
- **dstAddr (1 byte)**: Destination node identifier (0x01–0xFE, or 0xFF for broadcast).
- **messageId (2 bytes)**: Unique message sequence identifier. All fragments composing the same segmented message share this ID.

TABLE II
LAMP OPERATIONAL MODES ARCHITECTURE

	Unreliable Mode (Best-Effort, No ACKs)	**Reliable Mode (SACK-ARQ Feedback)**
Connectionless *(Stateless)*	Best-effort datagram streaming.	Bulk data transfer.
Connected *(Stateful, 3WHS)*	Session-tracked stream.	Negotiated parameter stream.

Byte 0	**Byte 1**	**Byte 2 – 3**	**Byte 4**	**Byte 5**	**Byte 6**	**Byte 7**	**Byte 8**
`srcAddr`	`dstAddr`	`messageId`	`totalChunks`	`chunkIndex`	`payloadSize`	`flags`	`protocolVersion`

Fig. 1. Byte layout of the compact 9-byte Logical Header incorporating Node Addressing.

TABLE III
LAMP OTA PACKET LAYOUT AND BYTE OFFSETS

Offset (Bytes)	**Field Name**	**Data Type**	**Size (Bytes)**
0	`srcAddr`	uint8_t	1
1	`dstAddr`	uint8_t	1
2–3	`messageId`	uint16_t	2
4	`totalChunks`	uint8_t	1
5	`chunkIndex`	uint8_t	1
6	`payloadSize`	uint8_t	1
7	`flags`	uint8_t	1
8	`protocolVersion`	uint8_t	1
9	`payload`	uint8_t[]	N (up to 244)
$9+N$	`crc`	uint16_t	2

- **`totalChunks` (1 byte)**: The total count of chunks into which the original large payload is divided.
- **`chunkIndex` (1 byte)**: The 0-based index of this specific fragment (0 to `totalChunks` - 1).
- **`payloadSize` (1 byte)**: The count of valid data bytes contained in the payload section ($N \leq 244$).
- **`flags` (1 byte)**: A control bitfield used to manage transmission and connection states:
  - `FLAG_ACK_REQ` (`0x04`): Selective Acknowledgment requested from receiver.
  - `FLAG_ACK` (`0x08`): SACK feedback packet containing binary bitmap of received chunks.
  - `FLAG_CONN_REQ` (`0x10`): Connection Request control frame.
  - `FLAG_CONN_ACK` (`0x20`): Connection Acknowledged control frame.
  - `FLAG_CONN_NACK` (`0x40`): Connection Refused control frame.
- **`protocolVersion` (1 byte)**: Identifies protocol version (currently 2) to ensure backward compatibility.

After the payload, a 16-bit CRC-16 (Modbus CRC-16, polynomial `0xA001`, initialized to `0xFFFF`) is appended. The checksum covers the entire 9-byte header and valid payload bytes, reliably detecting transmission corruptions.

D. Selective Acknowledgment (SACK) Retransmission

To provide reliable delivery under lossy radio conditions while maintaining minimal airtime overhead, the protocol implements a Selective Acknowledgment (SACK) mechanism [10].

1) Retransmission Handshake: When a transmission is executed in reliable mode:

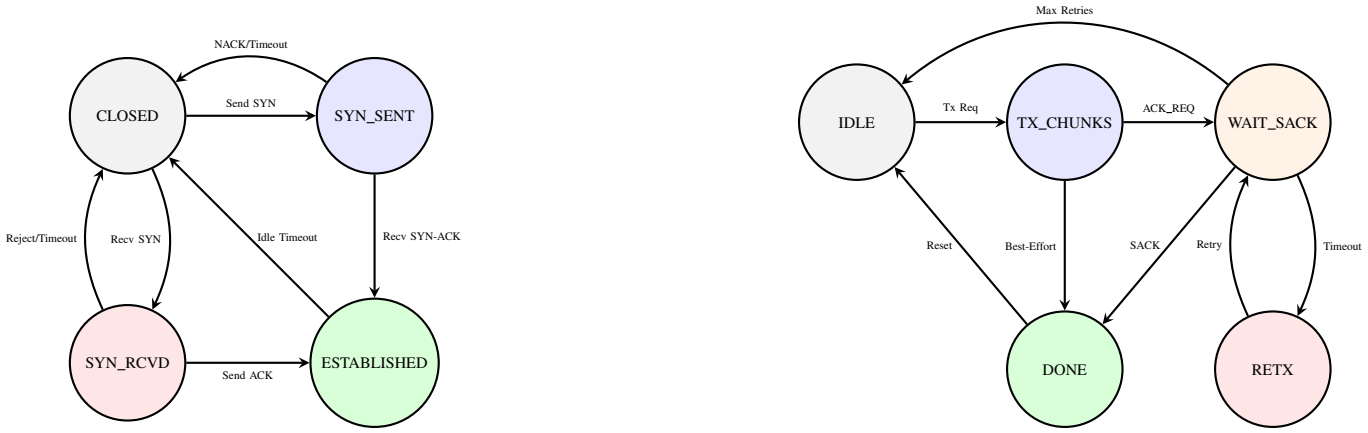
1) The transmitter segments the payload, serializes the chunks with `srcAddr` and `dstAddr`, and streams them sequentially. The final chunk index ($Index = \texttt{totalChunks} - 1$) is marked with the `FLAG_ACK_REQ` flag.
2) Upon receipt of the chunk requesting acknowledgment, the receiver evaluates its reassembly session. It identifies which fragments are still missing by checking its internal bitmask of received chunks.
3) The receiver responds by transmitting a SACK packet (`FLAG_ACK`) addressed back to the transmitter's `srcAddr`. The SACK's payload contains a binary bitmap where each bit represents the receipt status of a chunk (1 for received, 0 for missing).
4) The transmitter parses the received SACK bitmap. If some chunks are missing, it selectively retransmits only those lost chunks. The last retransmitted chunk has the `FLAG_ACK_REQ` flag set again to query the receiver.
5) This selective retry loop continues until the full message is reconstructed at the receiver, or the transmitter reaches the maximum retry count.

2) Half-Duplex Hardware Switching Guard: Because LoRa transceivers are half-duplex, a hardware transition period is required for the transmitter to switch from transmit (TX) mode to receive (RX) mode after sending a packet. If the receiver replies instantly, the transmitter will miss the SACK preamble. To guarantee correct reception, the receiver introduces a configurable hardware guard delay (e.g., 25 ms) before transmitting the SACK packet.

3) Timeout and Fallback Recovery: If the transmitter does not receive a SACK packet within a dynamic timeout threshold based on estimated ToA, it assumes that either the query or the response was lost. The transmitter falls back to retransmitting the final chunk of the sequence marked with `FLAG_ACK_REQ` to trigger a new status report. Up to 5 consecutive timeouts are allowed before the transmission is aborted as failed.

E. 3-Way Handshake (3WHS) and Connection Lifecycle

To support stateful stream negotiation and protect resource-constrained receivers from buffer overflows, `LAMP` Version 2 introduces a 3-Way Handshake (3WHS) mechanism.



(a) Connection Session Lifecycle FSM (Stateful vs. Stateless)

(b) Data Transfer & SACK-ARQ FSM (Reliable vs. Unreliable)

Fig. 2. Finite State Machine (FSM) specifications for connection session lifecycle and data transfer modes.

1) Handshake Procedure: As illustrated in Figure 3(a), the connection lifecycle follows a three-step exchange:

1) **SYN**: The initiator sends a `SYN` control frame (`FLAG_CONN_REQ`) addressed to the target node's `dstAddr`, containing a 4-byte `SynMetadata` payload (`requestedPayloadSize`, `windowSize`, `timeoutMs`).
2) **SYN-ACK**: Upon receiving the `SYN`, the peer evaluates available memory and protocol version. If accepted, it responds with a `SYN-ACK` control frame (`FLAG_CONN_REQ | FLAG_CONN_ACK`) addressed back to the initiator. The receiver clamps `acceptedPayloadSize` to its maximum buffer capability.
3) **ACK**: The initiator parses the `SYN-ACK`, adopts the negotiated payload size, and transmits a final `ACK` frame (`FLAG_CONN_ACK`). Both nodes transition their internal state machine to `ESTABLISHED`.

F. Robustness and Edge-Case Protections

To ensure protocol stability in multiagent and lossy environments, `LAMP` Version 2 introduces a 3-Way Handshake (3WHS) mechanism.

1) Session Establishment Handshake: Prior to streaming data in connected mode (Modes 3 and 4):

1) **SYN**: The initiator sends a `SYN` control frame (`FLAG_CONN_REQ`) to the target `dstAddr`, containing a 4-byte `SynMetadata` payload (`requestedPayloadSize`, `windowSize`, `timeoutMs`).
2) **SYN-ACK**: Upon receipt of the `SYN`, the receiver evaluates available memory and protocol version. If accepted, it responds with a `SYN-ACK` control frame (`FLAG_CONN_ACK`) containing a 4-byte `SynAckMetadata` payload confirming the negotiated parameters.
3) **ACK**: The initiator completes the handshake by transmitting an `ACK` control frame (`FLAG_ACK`) or immediately streaming the first data fragment with `chunkIndex` = 0.

Once the 3WHS is completed, both nodes transition to the `ESTABLISHED` state.

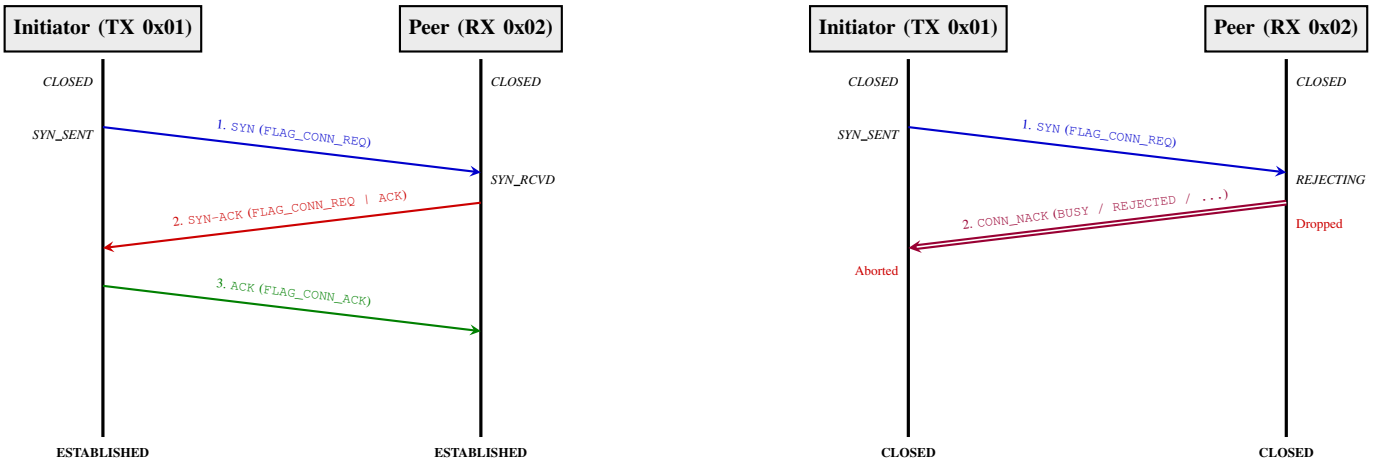
2) Edge-Case and Error Recovery Mechanics: To ensure protocol stability in multiagent and lossy environments, `LAMP` incorporates strict edge-case resolution mechanics:

- **Simultaneous SYN Resolution (Tie-Breaking)**: If two nodes transmit `SYN` frames to each other simultaneously, a deterministic tie-breaking rule resolves the conflict: the node with the lower numerical `NodeAddress` yields and transitions back to `CLOSED`, while the node with the higher `NodeAddress` proceeds with session setup.
- **Busy State Rejection**: If a receiver receives a `SYN` frame while its active reassembly session is already locked by another node, it responds with a `CONN_NACK` frame (`FLAG_CONN_NACK`) specifying a `BUSY` status code, causing the requester to back off.
- **Source Address Matching**: Initiators in `waitForSynAck` and `waitForSack` validate incoming control frames against the expected `srcAddr`, ignoring packets from unrelated third-party nodes.
- **Incomplete Handshake Timeout**: If a `SYN-ACK` or `ACK` of a 3WHS is lost in transit, the receiver in `SYN_RCVD` or initiator in `SYN_SENT` automatically aborts and resets to `CLOSED` after an inactivity timeout (`timeoutMs`).

IV. Preliminary Proof of Concept

An initial prototype used commercial Ebyte E220-series modules and validated the segmentation concept. That prototype demonstrated feasibility during preliminary field trials but highlighted three important limitations: (1) the modules exposed a proprietary network layer without full control of physical/link layer parameters, (2) an unoptimized, heavy application header in every chunk introduced excessive airtime overhead, and (3) there was no lightweight acknowledgment or retransmission mechanism.

To address these shortcomings, we designed the `LAMP` library on Semtech SX126x-class transceivers (demonstrated



(a) Successful 3-Way Handshake (3WHS)

(b) Connection Rejection (CONN_NACK) Failure Mode

Fig. 3. Sequence interaction flows for successful 3-Way Handshake and CONN_NACK rejection failure modes.

on a Heltec Wi-Fi LoRa 32 V3 development board). The new implementation keeps the protocol independent of vendor-specific APIs and focuses on the primitives described above: a 9-byte Logical Header (incorporating 8-bit node addressing and 3WHS dynamic negotiation), a per-packet CRC-16 covering header and payload, and SACK-based reliable delivery. Receivers reassemble messages by matching the `messageId` and collecting fragments indexed from 0 to `totalChunks` - 1. Duplicate fragments are ignored by checking the received bitmap in the session.

V. Hardware Level Implementation

To validate the proposed protocol, a complete implementation of the `LAMP` library was developed in C++17. The design is modular, separating the transport state machine from physical hardware dependencies.

A. Hardware Abstraction Layer (HAL)

To keep the protocol core hardware-agnostic, a clean Hardware Abstraction Layer interface (`RadioLibHal`) was implemented. On the physical nodes, the concrete class `EspHal` wraps the ESP32-S3 hardware resources:

- **SPI Communication**: Manages registers configuration and FIFO buffers transfer for the Semtech SX1262 transceiver.
- **GPIO Interrupts**: Maps physical pins, including Chip Select (NSS), Reset (RST), Busy status indicator (BUSY), and the hardware interrupt pin (DIO1) used to trigger RX/TX completion interrupts.
- **Timing Primitives**: Implements millisecond counters (`millis()`) and microsecond/millisecond delays (`delay()`).

B. Host-Based Unit Testing

Following a Test-Driven Development (TDD) methodology, the protocol core (e.g., packet parsing, reassembler sessions, and CRC checksums) was validated in isolation on a desktop host (Linux/macOS). The tests compile and run natively using the **Unity** testing framework integrated into PlatformIO:

- **Integrity Tests**: Confirmed the Modbus CRC-16 computation on headers and partial/padded payloads.
- **Reassembly State Machine**: Validated reassembly handling of out-of-order and duplicate chunks.
- **Stale Session Pruning**: Confirmed that incomplete reassembly buffers are correctly pruned after exceeding the timeout threshold.

C. Physical Benchmarking Setup

Physical tests were conducted using two Heltec Wi-Fi LoRa 32 V3 development boards. The transmitter node was connected to a console runner to sweep payload sizes, while the receiver node operated as a standalone listener. The transceivers were configured with the following parameters:

- **Carrier Frequency**: 868.0 MHz
- **Spreading Factor (SF)**: 7
- **Bandwidth (BW)**: 500.0 kHz
- **Coding Rate (CR)**: 4/5
- **Output Power**: 10 dBm
- **Preamble Length**: 8 symbols

VI. Proposed Experimental Methodology

To validate the reliability, latency, and throughput of the `LAMP` protocol, we outline a series of physical field experiments to be conducted under representative operational scenarios.

A. Experimental Parameters

The validation will sweep the following parameters:

- **Transmission Distance and Spatial Geometry**: Tests will be carried out at physical distances of 1, 10, 100, 250, 500, 1000, 2500, and 5000 meters. To account for spatial fading, antenna polarization discrepancies, and localized multi-path fading, testing in the short range (from 0 to 100 meters) will utilize a circular spatial sampling

strategy, gathering data points at multiple angular offsets around the transmitting antenna. For long-range setups (greater than 100 meters) where circular navigation is impractical, data collection will follow a linear path.

- **Propagation Conditions**: All distance steps will be benchmarked under both Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS) environments to test protocol performance under varying noise and packet corruption conditions.
- **Payload Sequence Size**: Payloads will scale from 1 chunk (representing the boundary condition) up to 100 chunks (approximately 24.6 KB total data size).
- **Radio Configuration**: Nodes will maintain standard settings (SF7, BW 500 kHz, CR 4/5, 10 dBm TX power) using the integrated Semtech transceivers.

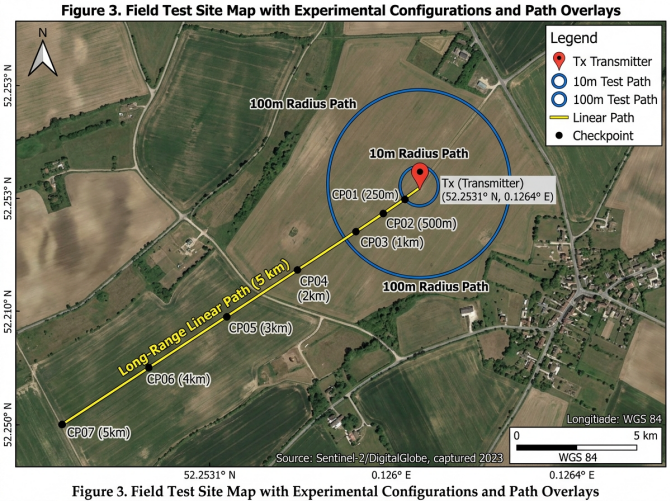


Fig. 4. Satellite view of the experimental testbed showing the transmitter node location, the circular spatial sampling points ($\leq$ 100 meters), and the linear range checkpoints ($>$ 100 meters) under Line-of-Sight (LoS) and Non-Line-of-Sight (NLoS) profiles.

B. Evaluation Metrics

For each combination of distance, environment, and payload size, 15 independent iterations will be performed. The following metrics will be collected and analyzed:

1) **Packet Delivery Ratio (PDR)**: The percentage of successfully reconstructed payloads at the receiver.
2) **End-to-End Latency**: The total duration elapsed from the initiation of split-packet serialization at the transmitter to the final payload reassembly callback at the receiver.
3) **Throughput (Goodput)**: The effective rate of application-level payload data delivered, calculated as $\text{PayloadSize (bits)}/\text{Latency (seconds)}$.
4) **Retransmission Overhead**: The count of extra chunks transmitted in reliable mode compared to the baseline case, representing the cost of SACK-driven error correction.

These benchmarks will allow us to evaluate the efficiency of the SACK retransmission handshake compared to best-effort unreliable streaming in lossy physical channels. The empty template for recording these empirical results is structured in Table IV.

VII. Conclusion and Architectural Roadmap

This paper presented the design, implementation, and hardware validation of `LAMP`, a lightweight and reliable transport-layer architecture optimized for transmitting segmented large payloads over point-to-point LoRa links. By replacing traditional Stop-and-Wait per-packet acknowledgments with selective retransmissions requested via cumulative binary bitmaps, and eliminating redundant SOM/EOM markers in favor of implicit index derivation, the protocol achieves high reliability under lossy conditions with minimal airtime overhead and duty-cycle footprint.

A. Three-Level Architectural Evolution Roadmap

To guide the long-term progression of `LAMP` from point-to-point data links to decentralized multiagent mesh networks, we define a strategic three-level architectural roadmap:

- **Level 1: Point-to-Point Addressed Communication (Implemented & Validated)**: The core protocol stack established in this work. It features 8-bit source and destination node addressing (`srcAddr`, `dstAddr`), dynamic 9-byte header packing, four operational modes (unreliable best-effort datagrams to stateful SACK-ARQ streams), a 3-Way Handshake (3WHS) for parameter negotiation, and hardware-validated edge-case resolution (BUSY rejection, address filtering, and simultaneous SYN tie-breaking).
- **Level 2: Multi-Node Local Area Communication (Near Future Extension)**: Extending the link layer to support local multi-node clusters sharing the same RF channel. Key features will include:
  - *Broadcast and Multicast Messaging*: Utilizing `0xFF` for unacknowledged broadcast telemetry and group addressing for multiagent synchronization.
  - *Shared Channel Access Control*: Implementing Channel Activity Detection (CAD) with randomized exponential backoff (CSMA-CA) or Time Division Multiplexing (TDM) [9] to prevent collisions during bulk packet bursts in multi-node environments.
  - *Adaptive Data Rate (ADR)*: Dynamically tuning Spreading Factor (SF7–SF12) and Coding Rate (CR) based on RSSI/SNR metrics embedded in SACK bitmaps.
- **Level 3: Multi-Hop / Multi-Agent Mesh Networking (Future Vision)**: Scaling the architecture to autonomous multi-hop networks, such as Unmanned Aerial Vehicle (UAV) swarms, distributed sensor meshes, and relay networks [7], [8]:
  - *Mesh Routing Protocol*: Integrating dynamic routing (e.g., AODV-Light or RPL adaptation) with hop-by-hop vs. end-to-end SACK reliability options to limit retransmission overhead over multi-hop links [8].
  - *IPv6 Network Layer Integration*: Leveraging the 8-bit Node ID mapping to establish full IPv6 link-local

TABLE IV
LAMP PLANNED EXPERIMENTAL BENCHMARKING RESULTS TEMPLATE

Distance	Path	Payload	PDR (%)		Avg Latency (ms)		Avg Goodput (Kbps)		Retransmission Overhead	
(m)	Type	Chunks	Case A	Case B	Case A	Case B	Case A	Case B	Chunks Sent	Retries
1	LoS	1–100								
	NLoS	1–100								
10	LoS	1–100								
	NLoS	1–100								
100	LoS	1–100								
	NLoS	1–100								
250	LoS	1–100								
	NLoS	1–100								
500	LoS	1–100								
	NLoS	1–100								
1000	LoS	1–100								
	NLoS	1–100								
2500	LoS	1–100								
	NLoS	1–100								
5000	LoS	1–100								
	NLoS	1–100								

capability (`fe80::/64`), connecting micro-drones directly to global IP infrastructure.

– *TDMA / Scheduled Time-Slotted Access*: Incorporating microsecond time synchronization for collision-free multiagent payload transfers in high-density networks.

DECLARATIONS

Data Availability Statement

All firmware implementations, C++17 library source code, host unit testing suites, and benchmarking scripts are available in the public project repository.

Ethical Statement and AI Tool Usage Disclosure

This study did not involve human participants or animal subjects. AI assistant tools were utilized strictly for code formatting, micro-typography auditing, and manuscript style calibration in compliance with IEEE publication ethics guidelines.

Conflict of Interest Statement

The authors declare that they have no competing financial or personal interests that could influence the work reported in this paper.

REFERENCES

[1] M. Radeta, J. Pestana, P. Abreu, R. Freitas, F. Silva, D. Vieira, R. Prieto, M. Fernandez, F. Alves, T. Dellinger, S. Neves, and E. Delory, "Triton—open telemetry and location estimation for marine monitoring based on iot and lora," *IEEE Journal of Oceanic Engineering*, vol. 50, no. 2, pp. 1244–1258, 2025.

[2] T. Elshabrawy and J. Al-Ali, "Lora technology demystified: From link behavior to cell capacity," *IEEE Transactions on Wireless Communications*, vol. 17, no. 10, pp. 6611–6621, 2018.

[3] F. Adelantado, X. Vilajosana, P. Tuset-Peiro, B. Martinez, J. Melia-Segui, and T. Watteyne, "Understanding the limits of lorawan implementations," *IEEE Communications Magazine*, vol. 55, no. 9, pp. 34–40, 2017.

[4] S. De, H. M. Jalajamony, S. Adhinarayanan, S. Joshi, H. Upadhyay, and R. Fernandez, "Multimedia transmission over lora networks for iot applications: A survey of strategies, deployments, and open challenges," *Sensors*, vol. 25, no. 23, p. 7128, 2025.

[5] P. J. Marcelis, V. S. Rao, and R. V. Prasad, "Dare: Data recovery through application layer coding for lorawan," in *Proceedings of the 2017 IEEE/ACM Second International Conference on Internet-of-Things Design and Implementation (IoTDI)*, 2017.

[6] T. Chen, D. Eager, and D. Makaroff, "Efficient image transmission using lora technology in agricultural monitoring iot systems," in *2019 IEEE International Conference on Internet of Things (iThings) and IEEE Green Computing and Communications (GreenCom) and IEEE Cyber, Physical and Social Computing (CPSCom) and IEEE Smart Data (SmartData)*, 2019, pp. 937–944.

[7] J. Kim, J. Jenkins, J. Seol, and M. Kwak, "Extending data transmission in the multi-hop lora network," *Issues in Information Systems*, vol. 23, no. 3, pp. 292–304, 2022.

[8] A. W.-L. Wong, S. L. Goh, M. K. Hasan, and S. Fattah, "Multi-hop and mesh for lora networks: Recent advancements, issues, and recommended applications," *ACM Computing Surveys*, vol. 56, no. 6, pp. 1–43, 2024.

[9] C.-C. Wei, J.-K. Huang, C.-C. Chang, and K.-C. Chang, "The development of lora image transmission based on time division multiplexing," in *2020 International Symposium on Computer, Consumer and Control (IS3C)*, 2020, pp. 323–326.

[10] A. Calabrò, E. Marchetti, and M. T. Paratore, "Evaluating use of arq strategies in communication protocols for search and rescue," in *Proceedings of the 21st International Conference on Web Information Systems and Technologies (WEBIST)*. SCITEPRESS, 2025, pp. 59–70.